

INDEX

Sr. No.	Title	Date	Marks	Sign.
1.	Prepare setup for Mininet network emulation environment with the help of Virtual box and Mininet. Demonstrate the basic commands in Mininet and emulate different custom network topology (Simple, Linear, and Tree). View flow tables.			
2.	After studying open-source POX and Floodlight controller, install controller and run custom topology using remote controller like POX and floodlight controller. Recognize inserted flows by controllers.			
3.	Create a SDN environment on Mininet and configure a switch to provide a firewall functionality using POX controller.			
4.	Using Mininet as an Emulator and POX controller, build your own internet router. Write simple router with a static routing table. The router will receive raw Ethernet frames and process the packet forwarding them to correct outgoing interface. You must check the Ethernet frames are received and the forwarding logic is created so packets go to the correct interface.			
5.	Emulate and manage a Data Centre via a Cloud Network Controller: create a multi-rooted tree-like (Clos) topology in Mininet to emulate a data center. Implement specific SDN applications on top of the network controller in order to orchestrate multiple network tenants within a data center environment, in the context of network virtualization and management.			
6.	Mini Project			

Experiment No. 1

Title:

Set up Mininet network emulation environment using Virtual Box and Mininet. Demonstrate the basic commands in Mininet and emulate different custom network topology (Simple, Linear, and Tree). View flow tables.

Aim:

To Prepare setup for Mininet network emulation environment with the help of Virtual box and Mininet. Demonstrate the basic commands in Mininet and emulate different custom network topology (Simple, Linear, and Tree). View flow tables.

Objectives:

- Access Mininet on the virtual machine
- Run the Ryu controller with a sample application
- Use various commands to gain experience with OpenFlow control of Open Switch

Theory:

Mininet emulator in Software Defined Networks: Mininet is a tool for software-defined networks. It is an emulator of a network and it is used to visualize the switches and application of software-defined networks in a virtualized environment. It is also used to test the software-defined network devices and those using OpenFlow protocols. The switches used in Mininet are OpenFlow switches.

Mininet is majorly used as a learning tool to test, experiment, and learn about software-defined networks. Mininet is preferable because it is very fast and helps us to create customizable topologies. It is also very easy to use.

Methods to Install Mininet:

1. Install Mininet as a virtual machine provided having a virtual box installed already

2. By installing it directly from the source.
3. By installing it from the package.
4. By upgrading the currently existing Mininet.

On Downloading the Mininet we get –

1.Command-line

:

The Mininet command line is similar to that of the Linux command line, using it we can interact with the system. First, we need to type the Linux commands on the shell prompt later we type the Mininet command on the Mininet CLI.

Mininet commands:

1. **nodes:** *It displays all the mininet nodes available*
2. **net:** *It displays the links between the nodes*
3. **dump:** *It displays the information about all the nodes.*

2.Mininet User interface:

Actually, there is no exact user interface for the Mininet because everything in the Mininet is controlled either by python scripts or commands. But the Mininet can be controlled using Graphical User Interface while creating and visualizing the topology in the graphic environment.

3. Python interfaces :

Mininet is available as a package and can be imported using python. We can also communicate and control the Mininet nodes by writing the scripts in python.

Features of Mininet :

1. Mininet provides us the space to develop and test our software-defined network applications without the need to set up a physical environment.
2. It gives us a network testbed thereby allowing us to develop and test applications that are using OpenFlow protocols.
3. It can be used without programming also.
4. It also gives us the flexibility to integrate python API, thereby paving the way for creating and experimenting with networks.
5. It also has a CLI(Command Line Interface) which is aware of topology and OpenFlow thereby allowing us to debug or run network tests for our application.

Advantages of Mininet :

1. It is very fast and takes very little time for booting.
2. It is easy to install and use.

3. It saves money because the emulators are cost-effective instead of testing with hardware devices.
4. It is also very easy to connect with real-world network devices.
5. It has high availability.

VirtualBox Mininet Setup

These instructions are for setting up a VM on your local machine for completing PA2.

Set up your VM

Mininet can create virtual networks with arbitrary topologies, which run within a Virtual Machine (VM) that you install at your laptop.

If you have any problems go to Mininet website for a tutorial and more detailed instructions.

- Download and install [VirtualBox](#), a virtualization platform.
- Download the [PA2 VM](#).
- **Enable host-only network:** On VirtualBox go to Preferences->Network->Host-only Networks and add a host-only network adapter (vboxnet0).
- **Import VM:** On VirtualBox, go to File->Import Appliance and select the .ova file from your unzipped folder. Follow the steps to create your VM.
- **Enable host-only adapter:** On your newly created VM, go to Settings->Network->Adapter 2 and add a host-only-network adapter (using vboxnet0).
- **Start and login in your VM.** When prompted, login using cs244/cs244 as user/password.
- **Check the available networks** in your VM by typing “ifconfig”.
 - You should see both eth0 and eth1. One should have a 192.168.X.X (host-only) and the other 10.0.X.X (NATted). If one of the two is missing, run “sudo dhclient ethX” (replace with eth0 or eth1).
 - Check again with ifconfig. Write down the IP address for host-only - it should be something like 192.168.56.101. You’ll use this to SSH to your VM shortly.
- **Check Internet connectivity:** Make sure you can access the Internet from your VM.
 - ping google.com
- The following should be installed for you, so skip this step.
 - Download a necessary tool for Python plotting:

- `sudo apt-get install python-matplotlib`
- Download pip, a Python package installation tool:
 - `sudo apt-get install python-pip`
- Download termcolor, a Python package for color formatting in terminal:
 - `pip install termcolor`
- **SSH into your VM:** Go back to your laptop. Open a terminal (Windows users try putty), and try to connect to your VM through ssh.
 - `ssh cs244@192.168.X.X` (replace with your host-only IP address)

By the end of this step you should be able to run the VM at your laptop, connect to it through SSH, and reach the Internet from within your VM.

Get familiar with Mininet

SSH into your VM. You can see a list with Mininet options using:

```
$ sudo mn -h
```

Start Mininet. By default, it will create a simple network with 2 hosts (h1, h2) and 1 switch (s1).

```
$ sudo mn
```

Now, while you are at the mininet CLI you can send commands to hosts. Start a ping from h1 to h2 using the following command. You should be seeing replies from 10.0.0.2:

```
mininet> h1 ping h2
```

iperf is a networking tool to test network performance. To see the maximum TCP throughput between the two hosts use the following command (NOTE: as h1 and h2 are connected locally within your VM you should get a very high bandwidth.)

```
mininet> iperf h1 h2
```

See what other commands are available through Mininet's CLI, and try a few of them.

```
mininet> help
```

Make sure you properly shutdown Mininet, otherwise it won't start correctly.

```
mininet> quit
```

If Mininet didn't shutdown correctly use the following to clean-up:

```
$ sudo mn -c
```

Experiment No-2

Title:

Studying open-source POX and Floodlight controller

Aim:

After studying open-source POX and Floodlight controller, install controller and run custom topology using remote controller like POX and floodlight controller. Recognize inserted flows by controllers.

Objectives:

- To study open-source POX and floodlight controller
- To understand working of custom topology

Theory:

Floodlight Controller:

Floodlight Controller is an SDN Controller developed by an open community of developers, many of which from Big Switch Networks, that uses with the Open Flow protocol to orchestrate traffic flows in a software-defined networking (SDN) environment. Open Flow is one of the first and most widely used SDN standards; it defines the open communications protocol in an SDN environment that allows the SDN Controller (brains of the network) to speak to the forwarding plane (switches, routers, etc.) to make changes to the network.

The SDN Controller is responsible for maintaining all of the network rules and providing the necessary instructions to the underlying infrastructure on how traffic should be handled. This enables businesses to better adapt to their changing needs and have better control over their networks.

The Floodlight Controller was originally offered by Big Switch as part of the Open Daylight Project, but in June 2013, Big Switch stepped away amid what seems to be conflicting interests with Cisco. While the Floodlight Controller is still open source, it is not involved with the Open Daylight Project.

Floodlight is a Java-based Open Flow controller platform. For more details and tutorials see the [FloodLight Open FlowHub page](#).

Install Prerequisites

Note: these instructions are a customized version of those on the [Floodlight Download page](#)

Make sure that you have Internet access:

```
ping -c1 www.stanford.edu
```

If not, ensure that each ethX interface on your VM has an IP assigned via DHCP:

```
ifconfig -a
```

Run for each interface without an IP assigned, replacing ethX as necessary:

```
sudo dhclient ethX
```

Update apt sources:

```
sudo apt-get update
```

Install prereqs:

```
time sudo apt-get install build-essential default-jdk ant python-dev
```

Pox Controller:

One option for the first exercise is to use POX. POX is a Python-based SDN controller platform geared towards research and education. For more details on POX, see [About POX](#) or [POX Documentation](#) on [NOXRepo.org](#).

We're not going to be using the reference controller anymore, which may still be running (do 'ps -A | grep controller' if you're unsure), so you should either press Ctrl-C in the window running the controller program, or kill it from the other SSH window:

```
$ sudo killall controller
```

You should also run `sudo mn -c` and restart Mininet to make sure that everything is "clean" and using the faster kernel switch. From your Mininet console:

```
mininet> exit
```



```
$ sudo mn -c
```

```
$ sudo mn --topo single,3 --mac --switch ovsk --controller remote
```

You then need to download the POX code from the [POX repository on github](http://github.com/noxrepo/pox) into your VM:

```
$ git clone http://github.com/noxrepo/pox
```

```
$ cd pox
```

OPTIONAL: The above should get you the latest release branch of POX. If you like the bleeding edge, you might like to switch to the latest development branch. This is documented on the POX website, but as of this writing, it's the "beta" branch:

```
$ git checkout beta
```

Now you can try running a basic hub example:

```
$/pox.py log.level --DEBUG misc.of_tutorial
```

This tells POX to enable verbose logging and to start the of_tutorial component which you'll be using (which currently acts like a hub).

The switches may take a little bit of time to connect. When an OpenFlow switch loses its connection to a controller, it will generally increase the period between which it attempts to contact the controller, up to a maximum of 15 seconds. Since the OpenFlow switch has not connected yet, this delay may be anything between 0 and 15 seconds. If this is too long to wait, the switch can be configured to wait no more than N seconds using the --max-backoff parameter. Alternately, you exit Mininet to remove the switch(es), start the controller, and then start Mininet to immediately connect.

The first line is from the portion of POX that handles OpenFlow connections. The second is from the tutorial component itself.

Create a custom topology shown as figure 1 and run in Mininet

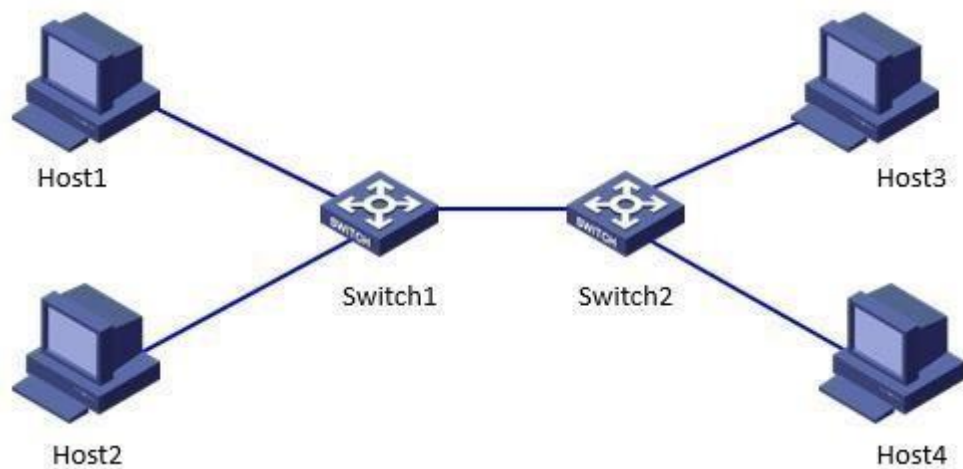


Figure 1

Create a python file

```
vi ~/mininet/custom/multi-hosts.py
from mininet.topo import Topo
class MyTopo( Topo ):
    "Simple topology example."
    def __init__( self ):
        "Create custom topo."

        # Initialize topology
        Topo.__init__( self )

        # Add hosts and switches
        Host1 = self.addHost( 'h1' )
        Host2 = self.addHost( 'h2' )
        Host3 = self.addHost( 'h3' )
        Host4 = self.addHost( 'h4' )
        Switch1 = self.addSwitch( 's1' )
        Switch2 = self.addSwitch( 's2' )
        # Add links
        self.addLink( Host1, Switch1 )
```

```

self.addLink( Host2, Switch1 )
self.addLink( Host3, Switch2 )
self.addLink( Host4, Switch2 )
self.addLink( Switch1, Switch2 )
topos = { 'mytopo': ( lambda: MyTopo() ) }

```

Start the topology and enter Mininet CLI
run

```
mininet@mininet-vm:~$ sudo mn --custom ./mininet/custom/multi-hosts.py --topo=mytopo
```

```

***Creating network
***Adding controller
***Adding hosts:
h1 h2 h3 h4
***Adding switches:
s1 s2
***Adding links:
(h1, s1) (h2, s1) (h3, s2) (h4, s2) (s1, s2) ***Configuring hosts
h1 h2 h3 h4
***Starting controller
c0
***Starting 2 switches
s1 s2
***Starting CLI:
mininet>
Display nodes

```

```

mininet> nodes
available nodes are:
c0 h1 h2 h3 h4 s1 s2

```

Display links

```

mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)

```

```
h3-eth0<->s2-eth1 (OK OK)
```

```
h4-eth0<->s2-eth2 (OK OK)
```

```
s1-eth3<->s2-eth3 (OK OK)
```

Display net

```
mininet> net
```

```
h1 h1-eth0:s1-eth1
```

```
h2 h2-eth0:s1-eth2
```

```
h3 h3-eth0:s2-eth1
```

```
h4 h4-eth0:s2-eth2
```

```
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:s2-eth3
```

```
s2 lo: s2-eth1:h3-eth0 s2-eth2:h4-eth0 s2-eth3:s1-eth3
```

```
c0
```

Display h1's interface configuration

```
mininet> h1 ifconfig
```

```
h1-eth0 Link encap:Ethernet HWaddr 8e:4f:d8:24:e1:e6
```

```
inet addr:10.0.0.1 Bcast:10.255.255.255 Mask:255.0.0.0
```

```
inet6 addr: fe80::8c4f:d8ff:fe24:e1e6/64 Scope:Link
```

```
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
```

```
RX packets:17 errors:0 dropped:0 overruns:0 frame:0
```

```
TX packets:8 errors:0 dropped:0 overruns:0 carrier:0
```

```
collisions:0 txqueuelen:1000
```

```
RX bytes:1326 (1.3 KB) TX bytes:648 (648.0 B)
```

```
Lo
```

```
Link encap:Local Loopback
```

```
inet addr:127.0.0.1 Mask:255.0.0.0
```

```
inet6 addr: ::1/128 Scope:Host
```

```
UP LOOPBACK RUNNING MTU:65536 Metric:1
```

```
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
```

```
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
```

```
collisions:0 txqueuelen:0
```

```
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

Experiment No- 3

Title

Create a SDN environment on Mininet and configure a switch to provide a firewall functionality

Aim:

To create a SDN environment on Mininet and configure a switch to provide a firewall functionality using POX controller.

Objectives:

- To study and understand SDN environment
- To implement a switch that provides firewall functionality
- To understand Pox controller

Theory:

Software Defined Networking and OpenFlow

Software-Defined Networking (SDN) is a recently proposed networking paradigm in which the data and control planes are decoupled from one another. One can think of the control plane as being the network's "brain", i.e., it is responsible for making all decisions, for example, how to forward data, while the data plane is what actually moves the data. In traditional networks, both the control- and data planes are tightly integrated and implemented in the forwarding devices that comprise a network. The SDN control plane is implemented by the "controller" and the data plane by "switches". The controller acts as the "brain" of the network, and sends commands ("rules") to the switches on how to handle traffic. OpenFlow has emerged as the de facto SDN standard and specifies how the controller and the switches communicate as well as the rules controllers install on switches.

Mininet

Mininet is a software stack that creates a virtual network on your computer/laptop. It accomplishes this task by creating host namespaces (h1, h2, etc) and connecting them through virtual interfaces. So when we run ping between the linux namespaces h1 and h2, the ping will

run from h1s namespace through a virtual interface pair created for h1 and h2, before it reaches h2. If h1 and h2 are connected through a switch as shown in the python code in the Mininet walkthrough, the ping will transit multiple virtual interface pairs. The switches that we will be using are running OpenVSwitch (OVS), a software-defined networking stack. Mininet will connect additional virtual interfaces between each virtual port on the switch with each connected host. The host name space allows each host to see the same file system, but operates as its own process that will run separately from each of the other host processes. The OVS version running on the Ubuntu image supports OpenFlow.

POX

Pox is a research/academic implementation of an OpenVFlow controller. It provides python hooks to program mininet-based software emulated networks.

Part 1: Mininet Primer

Learning Objectives (After this section, students will be able to...) :

- Use Mininet, a widely used self-contained network environment for research and experimentation, to build a test network topology.
- Use the basic debug tools built into Mininet to test connectivity between virtual hosts and get the state of the openflow switches in their topology.

Virtualbox and Vagrant Installation

Mininet is a tool for building emulated network topologies on a Linux host for experimentation with SDN. To ensure everyone is using the same environment and prevent you from messing up the networking on your computer, we'll run mininet inside of a virtual machine.

To manage the virtual machine, we'll be using a tool called **Vagrant**. Vagrant supports multiple "virtualization providers", but for compatibility with the Ubuntu image we'll use to run mininet, we suggest using **VirtualBox**. VirtualBox an open source and freely available virtualization system.

You will need to install both Vagrant and VirtualBox for this assignment. See their respective documentation for your platform for installation instructions:

- Vagrant
- VirtualBox

If you're using a flavor of Linux or MacOS, you may be able to install instead via package manager or brew, respectively.

Mininet VM Installation With Vagrant

Please follow the instructions [here](#) to set up the Vagrant VM for this project. This VM is based off Ubuntu 20.04 and includes a default set of mininet binaries and example scripts. Once you are inside the VM after running `vagrant ssh`, proceed to the next part.

There are a well known set of issues in using virtualization. If you encounter issues, reach out online and we'll see how we can help. Some common issues and answers are:

- Is your computer really old? It might not be able to be virtualized – talk to the staff via Ed.
- Is your computer really new, and runs on a non-x86 architecture? Talk to the staff via Ed.
- Make sure that virtualization is enabled in your BIOS.

For folks who would prefer to work in a remote VM, we're looking into this option, and should have more information soon!

For more information about vagrant, you can checkout.

Using Mininet

There is also a very helpful mininet community wiki with lots of good up-to-date information about how to use mininet effectively.

Inside of the mininet CLI, try running other commands like `help`, `net`, `nodes`, `links` and `dump`. Mininet starts with a default network that you can poke at. Find the mac address of the hosts, the ethernet ports connected, and the hostnames in the system.

Programming Mininet Topologies

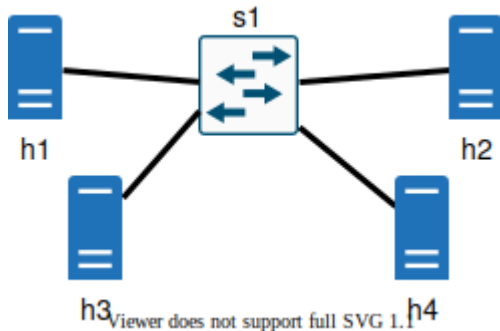
Mininet is also programmable using the python programming language. We have provided some sample topologies here, which should already be downloaded in the VM and unzipped into `~/project-1`.

In this directory, you'll find two different directories: `topo` and `pox`. Ignore the `pox` directory for now (it's used in part2). In the `topo` folder there are a variety of python files. These each define a topology for each of the following project portions. Run the project 1 file with `sudo`

python3 project-1/topos/part1.py. It will drop you into the CLI with the network topology defined in the python script.

Deliverables:

Your task in part one is to modify part1.py to represent the following network topology:



After creating the above architecture, your submission should include the following items:

- 1.(10 Points) Your modified topos/part1.py file
- 2.(10 points) Screenshots of the iperf h1 h4, dump, and pingall commands (from within the mininet cli in your running VM) in pdf format. These will help the TAs validate that your submission was working correctly on your machine if for some reason your submitted code does not work in the grading environment. Put your screenshots in the screenshots/part1 directory.

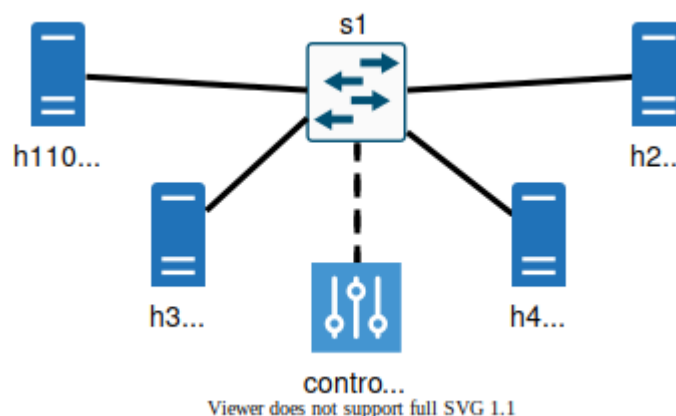
Part 2: SDN Controllers using POX

Learning Objectives (After this section, students will be able to...):

- Run their own POX openflow controller to add custom rules to switches in a mininet topology.
- Build an OpenFlow SDN controller program capable of connecting hosts together via a single programmable switch.
- Establish connectivity by flooding packets out switch ports.
- Create match → action rules to block or allow particular types of traffic through a switch based on layer 2 and layer 3 header information.

In part 1, we experimented with Mininet using its internal controller. In this (and future) parts, we will instead be using our own controller to send commands to the switches. We will be using the POX controller, which is written in Python.

For this assignment you will create a simple firewall using OpenFlow-enabled switches. The term “firewall” is derived from building construction: a firewall is a wall you place in buildings to stop a fire from spreading. In the case of networking, it is the act of providing security by not letting specified traffic pass through the firewall. This feature is good for minimizing attack vectors and limiting the network “surface” exposed to attackers. In this part, we will provide you with the Mininet topology, part2.py, to setup your network which assumes a remote controller listening on the default IP address and port number 127.0.0.1:6633. **You do not need to (and should not) modify this file.** The topology that this script will setup is as follows. Note that h1 and h4 are on the same subnet and a different one from h2 and h3.



For part 2, we will also provide you with a skeleton POX controller: a1part2controller.py. This file will be where you will make your modifications to create the firewall. The bootstrap should have setup a link from the ~/pox/pox/misc directory to ~/project-1/pox/a1part2controller.py. If no link is present at ~/pox/pox/misc/a1part2controller.py, you can create a new one by running `ln -s ~/project-1/pox/* ~/pox/pox/misc/`. You can then launch the controller with the command `sudo ~/pox/pox.py misc.a1part2controller`. Once the controller is running, you can run mininet with `sudo python3 ~/project-1/topos/part2.py`.

Using a terminal multiplexer like tmux or screen could be handy for running your controller in a separate terminal window within the VM. Alternatively you can just open multiple SSH connections too :)

The rules s1 will need to implement are as follows:

src ip	dst ip	protocol	action
any ipv4	any ipv4	icmp	accept

src ip	dst ip	protocol	action
any	any	arp	accept
any ipv4	any ipv4	-	drop

Basically, your Firewall should allow all ARP and ICMP traffic to pass. However, any other type of traffic should be dropped. It is acceptable to flood the allowable traffic out all ports. Be careful! Flow tables match the rule with highest priority first, where priority is established based on the order rules are placed in the table. When you create a rule in the POX controller, you need to also have POX “install” the rule in the switch. This makes it so the switch “remembers” what to do for a few seconds. **Do not handle each packet individually inside of the controller! Hint: To do this, look up ofp_flow_mod.** The [OpenFlow tutorial](#) (specifically " Sending OpenFlow messages with POX") and the [POX Wiki](#) are both useful resources for understanding how to use POX.

The Ed board is a great place for swapping resources and asking questions about how to get POX and Mininet working. The goal is to learn networking, not suffer in silence while fighting with a new tool :)

Experiment No- 4

Title:

Using Mininet as an Emulator and POX controller, build your own internet router. Write simple outer with a static routing table. The router will receive raw Ethernet frames and process the packet forwarding them to correct outgoing interface. You must check the Ethernet frames are received and the forwarding logic is created so packets go to the correct interface.

Aim:

To your own internet router using Mininet as an Emulator and POX controller, build. Write simple outer with a static routing table. The router will receive raw Ethernet frames and process the packet forwarding them to correct outgoing interface. You must check the Ethernet frames are received and the forwarding logic is created so packets go to the correct interface.

Introduction:

In this lab assignment you will be writing a simple router with a static routing table. Your router will receive raw Ethernet frames. It will process the packets just like a real router, then forward them to the correct outgoing interface. We'll make sure you receive the Ethernet frames; your job is to create the forwarding logic so packets go to the correct interface.

Your router will route real packets from a emulated host (client) to two emulated application servers (http server 1/2) sitting behind your router. The application servers are each running an HTTP server. When you have finished the forwarding path of your router, you should be able to access these servers using regular client software. In addition, you should be able to ping and traceroute to and through a functioning Internet router. A sample routing topology is shown below:

If the router is functioning correctly, all of the following operations should work:

- Pinging from the client to any of the router's interfaces (192.168.2.1, 172.64.3.1, 10.0.1.1).
- Tracerouting from the client to any of the router's interfaces
- Pinging from the client to any of the app servers (192.168.2.2, 172.64.3.10)
- Tracerouting from the client to any of the app servers
- Downloading a file using HTTP from one of the app servers

Get Started

Note: Please run the following commands as root on the VM.

Environment Setup

```
cd ~
wget
http://www.cs.princeton.edu/courses/archive/spr13/cos461/assignments/router/ass2.tar.gz
tar -xzvf ass2.tar.gz
cd ass2/
./config.sh
```

Configuration Files

There are two configuration files.

~/ass2/IP_CONFIG: Listed out the IP addresses assigned to the emulated hosts.

~/ass2/router/rtable (also linked to ~/ass2/rtable): The static routing table used for the simple router.

The default *IP_CONFIG* and *rtable* should look like the following:

```
> cat ~/ass2/IP_CONFIG
server1 192.168.2.2
server2 172.64.3.10
client   10.0.1.100
sw0-eth1 192.168.2.1
sw0-eth2 172.64.3.1
sw0-eth3 10.0.1.1
> cat ~/ass2/rtable
10.0.1.100 10.0.1.100 255.255.255.255 eth3
```

```
192.168.2.2 192.168.2.2 255.255.255.255 eth1
172.64.3.10 172.64.3.10 255.255.255.255 eth2
```

Test Connectivity of Your Emulated Topology

Start Mininet emulation by using the following command

```
> cd ~/ass2/
> export PYTHONPATH=$PYTHONPATH:/usr/lib/python2.7/site-
packages:/usr/lib/python2.6/site-packages:~/ass2/pox_module/
> ./run_mininet.sh
```

You should be able to see some output like the following:

```
*** Shutting down stale SimpleHTTPServers
*** Shutting down stale webservers
server1 192.168.2.2
server2 172.64.3.10
client 10.0.1.100
sw0-eth1 192.168.2.1
sw0-eth2 172.64.3.1
sw0-eth3 10.0.1.1
*** Successfully loaded ip settings for hosts
{'server1': '192.168.2.2', 'sw0-eth3': '10.0.1.1', 'sw0-eth1': '192.168.2.1',
'sw0-eth2': '172.64.3.1', 'client': '10.0.1.100', 'server2': '172.64.3.10'}
*** Creating network
*** Creating network
*** Adding controller
*** Adding hosts:
client server1 server2
*** Adding switches:
sw0
*** Adding links:
(client, sw0) (server1, sw0) (server2, sw0)
*** Configuring hosts
client server1 server2
*** Starting controller
*** Starting 1 switches
sw0
*** setting default gateway of host server1
server1 192.168.2.1
*** setting default gateway of host server2
server2 172.64.3.1
*** setting default gateway of host client
client 10.0.1.1
*** Starting SimpleHTTPServer on host server1
*** Starting SimpleHTTPServer on host server2
*** Starting CLI:
mininet>
```

Keep this terminal open, as you will need the mininet command line for debugging. Now, use another terminal to continue to the next step. (Do not do ctrl-z)

Mininet requires a controller, which we implemented in POX. To run the controller, use the following command:

```
> cd ~/ass2/
> ./run_pox.sh
```

You should be able to see some output like the following:

```
POX 0.0.0 / Copyright 2011 James McCauley
DEBUG:.home.ubuntu.ass2.pox_module.cos461.ofhandler:*** ofhandler: Successfully
loaded ip settings for hosts
{'server1': '192.168.2.2', 'sw0-eth3': '10.0.1.1', 'sw0-eth1': '192.168.2.1',
'sw0-eth2': '172.64.3.1', 'client': '10.0.1.100', 'server2': '172.64.3.10'}

INFO:.home.ubuntu.ass2.pox_module.cos461.srhandler:created server
DEBUG:.home.ubuntu.ass2.pox_module.cos461.srhandler:SRServerListener listening
on 8888
DEBUG:core:POX 0.0.0 going up...
DEBUG:core:Running on CPython (2.7.3/Aug 1 2012 05:14:39)
INFO:core:POX 0.0.0 is up.
This program comes with ABSOLUTELY NO WARRANTY. This program is free software,
and you are welcome to redistribute it under certain conditions.
```

```
Type 'help(pox.license)' for details.
DEBUG:openflow.of_01:Listening for connections on 0.0.0.0:6633
Ready.
POX>
```

Please note that you have to wait till Mininet is able to connect to the POX controller before you continue to the next step.

Once Mininet is connected, you will see the following output:

```
INFO:openflow.of_01:[Con 1/249473472573510] Connected to e2-e5-11-b6-b0-46
DEBUG:.home.ubuntu.ass2.pox_module.cos461.ofhandler:Connection [Con
1/249473472573510]
DEBUG:.home.ubuntu.ass2.pox_module.cos461.srhandler:SRServerListener catch
RouterInfo even, info={'eth3': ('10.0.1.1', '86:05:70:7e:eb:56', '10Gbps', 3),
'eth2': ('172.64.3.1', 'b2:9e:54:d8:9d:cd', '10Gbps', 2), 'eth1':
('192.168.2.1', '36:61:7c:4f:b6:7b', '10Gbps', 1)}, rtable=[]Keep POX
running. Now, open yet another terminal to continue to the next
step. (Dont' do ctrl-z)
```

Now you are ready to test out the connectivity of the environment setup. To do so, run the binary file of the solution “sr_solution”

```
> cd ~/ass2/
> ./sr_solution
```

You should be able to see some output like the following:

```
Loading routing table from server, clear local routing table.
Loading routing table
```

```
-----
Destination      Gateway           Mask             Iface
10.0.1.100        10.0.1.100        255.255.255.255 eth3
192.168.2.2        192.168.2.2        255.255.255.255 eth1
172.64.3.10        172.64.3.10        255.255.255.255 eth2
-----
```

```
Client ubuntu connecting to Server localhost:8888
Requesting topology 0
successfully authenticated as ubuntu
Loading routing table from server, clear local routing table.
Loading routing table
```

```
-----
Destination      Gateway           Mask             Iface
10.0.1.100        10.0.1.100        255.255.255.255 eth3
192.168.2.2        192.168.2.2        255.255.255.255 eth1
172.64.3.10        172.64.3.10        255.255.255.255 eth2
-----
```

```
Router interfaces:
eth3      HWaddr86:05:70:7e:eb:56
          inet addr 10.0.1.1
eth2      HWaddrb2:9e:54:d8:9d:cd
          inet addr 172.64.3.1
eth1      HWaddr36:61:7c:4f:b6:7b
          inet addr 192.168.2.1
<-- Ready to process packets -->
```

Conclusion:

In this way we have implemented the practical of router using the POX.

Experiment No- 5

Title:

Emulate and manage a Data Center via a Cloud Network Controller: create a multi-rooted tree-like (Clos) topology in Mininet to emulate a data center. Implement specific SDN applications on top of the network controller in order to orchestrate multiple network tenants within a data center environment, in the context of network virtualization and management.

Aim:

To Emulate and manage a Data Center via a Cloud Network Controller: create a multi-rooted tree-like (Clos) topology in Mininet to emulate a data center. Implement specific SDN applications on top of the network controller in order to orchestrate multiple network tenants within a data center environment, in the context of network virtualization and management.

Introduction:

In this assignment you will acquire a practical view of the concepts of network virtualization and cloud management in the context of data center environments. In particular, you will write SDN applications on top of an OpenFlow controller (POX) to implement routing, enforce firewall policies and provide support for Virtual Machine migration over a data center-like topology. This controller will essentially act as a network controller for a small cloud facility.

In general, network virtualization permits to share physical network resources so as to form diverse virtual networks (slices) on top of the infrastructure: each of these slices has e.g., its dedicated bandwidth allocation, broadcast domain, etc. This concept simplifies network management and increases the resource utilization of the physical infrastructure, allowing a cloud provider to amortize the costs of the underlying network. Mechanisms related to network virtualization are commonly used in data centers to orchestrate their multiple applications - servers - tenants. SDN in general and OpenFlow in particular can be used to implement such mechanisms, as you have learned in the lecture.

Emulate a Data Center and Manage it via a Cloud Network Controller

In this exercise, your first task is to create a multi-rooted tree-like (Clos) topology in Mininet to emulate a data center (see Figures 1 and 2). Your second task is to implement specific SDN applications on top of the network controller in order to orchestrate multiple network tenants within a data center environment, in the context of network virtualization and management. In Figure 1, we see a sample data center network design with a number of Virtual Machines (VMs) being hosted by distinct VM end-host servers. For simplicity, we assume that each server hosts exactly one VM instance (in practice they can of course support tens of VMs). In Figure 1, each set of VMs with the same color belongs to separate data center tenants; tenants may represent different application environments (different colors respectively) that the VMs host. For example, one tenant may be an enterprise client running its big data analytics, while another tenant may be a research group running simulations for an upcoming publication. In this assignment you will not need to virtualize your own hosts, since we make the convention that each individual physical host supports a single VM or virtual host; Mininet takes care of emulating these virtual hosts. It is assumed that each application is pre-installed in each host, thus you will not need to deal with the application part within a host. The goal is to become familiar with how the virtualized data center network supports and orchestrates the applications running on top of its infrastructure.

A properly designed data center network should efficiently route traffic across its rich interconnection fabric, satisfy isolation constraints, and provide VM migration capabilities for its tenants, among other requirements that you have learned about in the lecture. For example, routing strategies could exploit the rich topology of the data center, which contains multiple redundant paths between any pair of hosts, and choose the shortest path (in terms of number of hops) thus minimizing the allocated bandwidth on the network links.

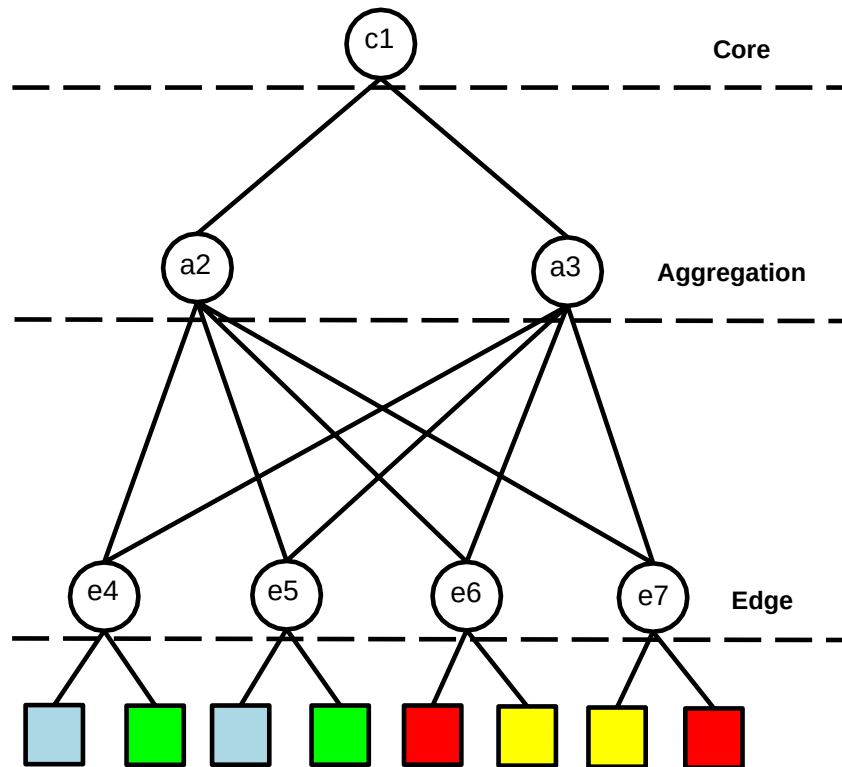


Fig. Sample layered data center topology

According to the isolation requirement, VMs belonging to the same tenant –offering the same service– should be able to communicate only with each other. Inter-VM communication between different tenants should be blocked. This is required for security reasons, as tenants generally require to operate within their private broadcast domains. Regarding the migration part, let's assume that a tenant's host is planned to shut down for maintenance purposes. In this case, the hosted VM has to move into a different host, i.e., migrate, so as to be able to continue offering the service. Such VM transfers have to be done with minimal reconfiguration and service disruption within the network, assuming that the selected destination hosts have the available resources to host the migrated VMs. For example, assume that the green host h2 in Figure 1 is planned to go down. Red hosts h5 and h8 are both available (we assume that they do not belong to any tenant). Therefore, a possible migration choice is to move the VM from host h2 to host h5. For the exercise you do not need to take care of saving, migrating and restoring the VM state from the end-host's perspective; instead, you only need to take into account how the network treats the migration of traffic destined from the old VM to the new VM, since the migration should be transparent on the IP layer.

In the following sections we describe the steps towards satisfying the aforementioned requirements in your network topology, using the network controller, so please follow each step carefully. Part A will take you through the processes of building the network topology, performing shortestpath routing and implementing and configuring a firewall application.

Conclusion:

In this way we have implemented the practical for emulate and manage a data center via a cloud network controller.